DRAFT: Decoupling Backpropagation from Pre-trained Backbone for Efficient Transformer Fine-Tuning on Edge

Zhirui Huang†, Shiwei Liu†, Haozhe Zhu†, Qi Liu†, and Chixiao Chen†‡*
† *State Key Laboratory of Integrated Chips and Systems (SKLICS), Fudan University, Shanghai, China*
‡ *Shaoxin Laboratory, Shaoxing, Zhejiang, China*
* E-mail: cxchen@fudan.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132748

***Abstract*—Transformers have demonstrated outstanding performance across diverse applications recently, necessitating fine-tuning to optimize their performance for downstream tasks. However, fine-tuning remains challenging due to the substantial computational costs and storage overhead of backpropagation (BP). The existing fine-tuning techniques require the BP through the massive pre-trained backbone weights for computing the input gradient, resulting in significant computing overhead and memory footprint for resource-constrained edge devices. To address the challenge, this work proposes an algorithm-hardware co-design framework, DRAFT, for efficient Transformer fine-tuning by decoupling the BP from the backbone weights, thereby efficiently reducing the BP overhead. The framework employs Feedback Decoupling Approximation (*FDA*), an efficient fine-tuning algorithm that decouples BP into two low-complexity pathways: trainable adapter pathway and sparse ternary Bypass Network (*BPN*) pathway. The two pathways work collaboratively to approximate the conventional BP process. Further, a DRAFT accelerator is proposed, featuring a reconfigurable design with lightweight sparse gather networks and dynamic workflows to fully harness the sparsity and data parallelism inherent to the FDA. Experimental results demonstrate that DRAFT achieves a speedup of 4.9× and an energy efficiency improvement of 4.2× on average compared to baseline fine-tuning methods across multiple fine-tuning tasks with negligible accuracy loss.**

I. Introduction

In recent years, Transformers [1] have demonstrated outstanding performance on tasks of multiple domains, including Natural Language Processing (NLP) [2], [3] and Computer Vision (CV) [4]. Owing to their impressive capabilities, it is prevalent to fine-tune the pre-trained Transformers (referred to as ***backbone***) for various downstream tasks. The demand for local fine-tuning on edge hardware is rapidly growing. This enhances user privacy protection and reduces latency and energy consumption associated with data transmission.

The primary challenge of Transformer fine-tuning is that backpropagation (BP) nearly tripled the computational cost compared to feedforward (FF)-only inference due to the BP computation for both weight and input gradient [5], [6]. Unlike conventional full fine-tuning, which updates all the pre-trained weight as illustrated in Fig. 1(a), Parameter-Efficient Fine-Tuning (PEFT) [7]–[9] is a recent research direction aims to limit the number of trainable parameters for online learning. For instance, as illustrated in Fig. 1(a), Adapters [7] are small bottleneck modules inserted into Transformer layers, while LoRA [10] injects trainable parameters alongside the frozen backbone weights. They reduce the associated weight gradient computation and optimizer state storage efficiently.

However, Transformer fine-tuning remains challenging for two main reasons. First, FF accuracy is critical for effective weight updates, restricting the compression feasible for the pre-trained model prior to the fine-tuning process [11], [12]. As a result, the BP through the backbone, comprising millions to billions of parameters, incurs substantial computational overhead. Second, as shown by the red line in Fig. 1(a), existing optimizations primarily address the weight gradient computation yet overlook the input gradient computation, necessitating BP through the heavy but frozen backbone weight. This oversight limits the reduction of fine-tuning computation to a maximum of 31%, as illustrated in Fig. 1(b).

The reliance of BP on the backbone weight makes Transformer fine-tuning computationally intensive for resource-limited edge devices. Previous efficient DNN training accelerators based on asymmetric BP [13], [14] and sparse BP [15], [16] suggest that accurate, symmetric BP may not be essential. Inspired by this line of research, we investigate the potential to uniformly reduce the BP computation cost for both input and weight gradients across all linear layers in the Transformer model, thereby significantly improving fine-tuning efficiency.

We propose an algorithm-hardware co-design framework for edge Transformer fine-tuning called DRAFT. As shown in Fig. 2, the key idea of DRAFT is to decouple the BP through the backbone weight into two lightweight pathways. These pathways jointly approximate pre-trained weight with low complexity, eliminating BP through the backbone weight without trading off fine-tuning accuracy. The major contributions are summarized as follows:

- We propose Feedback Decoupling Approximation (*FDA*), an efficient fine-tuning technique that fully reduces BP overhead by skipping the BP through the backbone weight with a sparse ternary Bypass Network (*BPN*). Decoupled-initialized and trainable adapters bridge the separated FF and BP pathways, reducing the BPN's complexity while maintaining fine-tuning performance.

* Chixiao Chen is the corresponding author.

This work was supported by Shanghai 2024 "Science and Technology Innovation Action Plan" Fundamental Research Program in Integrated Circuit under Grant No. 24JD1400300 and the National Natural Science Foundation of China under Grant No. 62322404.

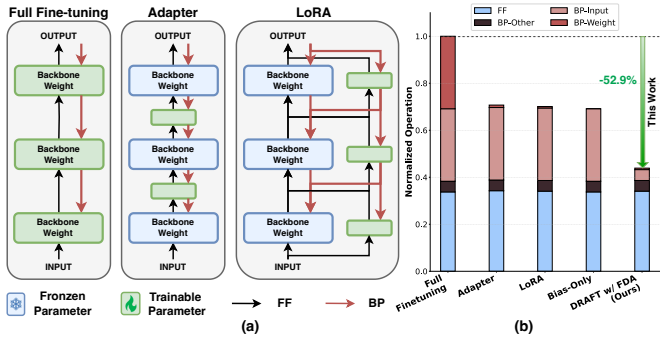

Fig. 1. (a) Illustration of the fine-tuning path of different fine-tuning methods with Full Fine-tuning, Adapter, and LoRA. (The compute paths of the linear layers are illustrated while the attention mechanism propagates primitively). (b) Fine-tuning operations comparison of Full Fine-tuning, Adapter [7], LoRA [10], Bias-only [8], and DRAFT (based on FDA) for the BERT-Base model.

- We design an efficient DRAFT accelerator for Transformer fine-tuning to leverage the sparsity and data parallelism inherent to the FDA. The reconfigurable processing element array supports lightweight gather networks and dynamic workflow to improve the throughput across different fine-tuning phases dynamically.
- We evaluate DRAFT on various models, datasets, fine-tuning algorithms, and hardware accelerators. Evaluation results illustrate that the FDA reduces the computational overhead of BP to below 15% with negligible accuracy loss. The algorithm-hardware co-design enables DRAFT to further reduce fine-tuning latency by 4.9× and substantially improve energy efficiency by 4.2× on average compared to the baselines.

II. Background and Motivation

A. Efficient Transformer Fine-tuning

Recently, numerous efficient fine-tuning techniques have been introduced to enable fine-tuning Transformers on commercial GPUs and even edge devices. A prominent parameter-efficient fine-tuning (PEFT) technique, LoRA [10], reduces the number of trainable parameters by integrating the trainable adapter along the linear layer. The computation output y of a linear layer on input X in LoRA is:

$$y = (W_o + \Delta W)X \approx (W_o + \frac{\alpha}{r} \cdot AB^\top)X \tag{1}$$

where W_o is the pre-trained weight matrix, ΔW is the weight update to W_o, and A and B are the low-rank approximation of ΔW initialized with random data and zeros, with a scaling factor of α and rank r. LoRA achieves near full fine-tuning accuracy by updating only a small fraction of the weights. However, prior efficient fine-tuning solutions [17]–[19] based on PEFT fall short of addressing the BP overhead associated with computing input gradients via the enormous backbone weight, which will be further discussed in Sec. II-C.

B. Training Techniques beyond Backpropagation

Substantial research has explored alternative training techniques to replace traditional BP. For example, the Google Brain team introduced Feedback Alignment [20], [21], which addresses the weights symmetry problem by proposing *asymmetric backpropagation*. This line of research focuses on producing high-quality gradients to achieve comparable performance to the conventional BP algorithm rather than exploring acceleration opportunities. Additionally, some strategies approximate weight gradients directly [22]. For instance, ADA-GP [5] predicts BP gradients using an auxiliary neural network. These techniques reveal that precise gradient computation via the chain rule is not always essential. However, most methods have primarily been validated on small-scale neural networks and suffer from significant accuracy loss when applied to larger, more complex models [21], [22].

C. Motivation

Backpropagation Computation Overhead: We begin by analyzing the network structure as a multi-layer perceptron (MLP): $Y = f_N(f_{N-1}(...f_2(f_1(x))...))$, where the output at layer i is given by $y_i = \mu_i(a_i) = \mu_i(W_i X_i)$. W_i is the pre-trained backbone weight for i-th layer, X_i denotes the input, and μ_i is the activation function, with the bias term omitted for simplicity. As illustrated in Fig. 3(a), the major computation overhead of conventional BP is to obtain the gradients of the input and the weights based on the loss function L:

$$\frac{\partial L}{\partial X_i} = \frac{\partial L}{\partial y_i}\frac{\partial y_i}{\partial a_i}\frac{\partial a_i}{\partial X_i} = \frac{\partial L}{\partial y_i}\mu_i' W_i^\top \tag{2}$$

$$\frac{\partial L}{\partial W_i} = \frac{\partial L}{\partial y_i}\frac{\partial y_i}{\partial a_i}\frac{\partial a_i}{\partial W_i} = \frac{\partial L}{\partial y_i}\mu_i' X_i \tag{3}$$

Current fine-tuning algorithms primarily optimize by reducing the number of trainable weights, thus decreasing the computation associated with the weight gradient term $\frac{\partial L}{\partial W_i}$ in Eq. (3) (denoted as G_W in Fig. 3). However, they fall short of addressing the computation of the input gradient $\frac{\partial L}{\partial X_i}$ in Eq. (2) (denoted as G_X in Fig. 3). Consequently, as illustrated in Fig.1 (b), BP computation costs remain nearly equivalent to those of the FF stage. Techniques such as Partial Tuning [23], [24] mitigate this issue by shortening the BP depth and tuning

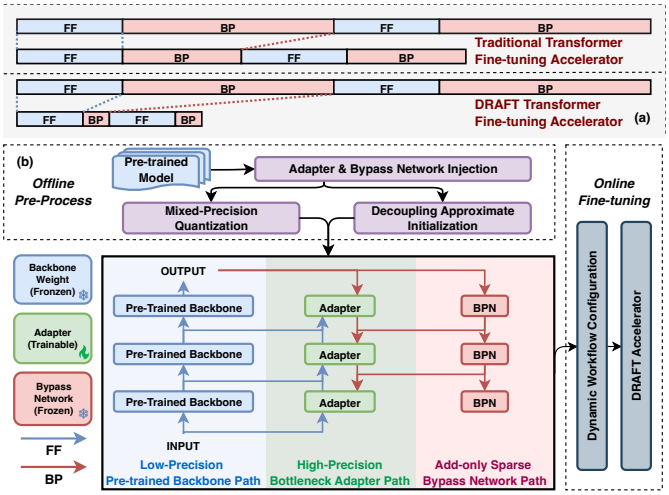

Fig. 2. (a) Comparison of DRAFT design concept with previous fine-tuning accelerator. (b) DRAFT framework overview, where the fine-tuning FF and BP pathways are decoupled and asymmetric compared to conventional BP.

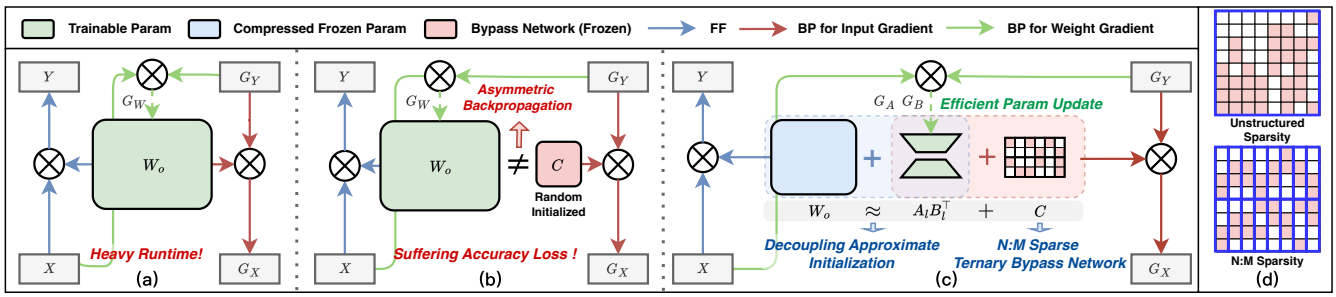

Fig. 3. The linear layer computation graph comparison of (a) conventional backpropagation, (b) asymmetric backpropagation based on Feedback Alignment, (c) approximate backpropagation based on Feedback Decoupling Approximation (FDA). The FDA delivers an accuracy-cost trade-off solution by decomposing and decoupling approximating the pre-trained weight. (d) Illustration of unstructured sparsity and N:M sparsity where N:M sparsity is more hardware-friendly.

the final few layers. However, these approaches limit updates to only a small subset of Transformer blocks, constraining the potential performance improvements. **In contrast, DRAFT fully reduces BP computation cost for input and weight gradient** by decoupling the BP from the backbone weight, striking a balance between hardware and algorithm efficiency.

III. Overview of DRAFT Framework

In this work, we propose DRAFT, an efficient algorithm-hardware co-design framework for Transformer fine-tuning on edge. The fine-tuning process based on DRAFT is illustrated in Fig. 2(b). **In the offline stage**, DRAFT pre-processes the pre-trained model in three steps: (1) Adapter and Bypass Network (*BPN*) Injection, (2) Pre-trained Model Compression, and (3) Decoupling Approximate Initialization. Based on the Feedback Decoupling Approximation (*FDA*) algorithm, DRAFT introduces an asymmetric fine-tuning pathway that skips the BP through the backbone weight with lightweight BPNs. The adapters bridge the gap between the separate FF and BP pathways, preventing accuracy degradation from the asymmetric BP, as detailed in Sec. IV-A. The BPN is an N:M sparse ternary network designed for both hardware and algorithm efficiency, as detailed in Sec. IV-B. **In the online stage,** DRAFT divides fine-tuning into three pathways. During the FF phase, the output is computed through the compressed frozen backbone and the trainable adapter. In the BP phase, the error gradient propagates through the BPN and adapter with minimum computation overhead. The reconfigurable DRAFT accelerator is designed to efficiently leverage the sparsity and parallelism inherent to the FDA, as detailed in Sec. V.

IV. DRAFT Fine-tuning Algorithm

A. Feedback Decoupling Approximation Algorithm

Asymmetric BP based on the Feedback Alignment [20], [21] has long been studied to produce synthetic gradients instead of conventional BP, eliminating the reliance on weight symmetry of BP. As illustrated in Fig. 3(b), this approach demonstrates the potential to bypass the BP through the backbone network by reformulating the input gradient computation in Eq. (2):

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial y}\mu' C \tag{4}$$

In the conventional BP algorithm, $C = W_o^\top$, which is also referred to as *symmetric BP*. However, asymmetric BP uncovers redundancy in the standard approach, relaxing the constraint of weight symmetry, i.e. $C \neq W_o^\top$, thereby presenting an opportunity to perform BP without the backbone weight. Here, C can be a compressed version of the weight W_o or even random numbers, expressed as $C = \phi(W_o)$, where $\phi(\cdot)$ can be different compression algorithms or random number generators [21]. However, asymmetric BP is inherently prone to accuracy degradation as it lacks the ability to leverage pre-trained knowledge.

Feedback Decoupling Approximation for Lightweight Backpropagation: As illustrated in Fig. 3(c), to address the performance degradation of vanilla asymmetric BP, we propose the Feedback Decoupled Approximation (FDA) algorithm, which compensates for compression loss during the Bypass Network initialization with a trainable adapter. Different from the asymmetric BP, the FDA initializes the BPN by decoupling the backbone weight into two components:

$$\begin{aligned}\frac{\partial L}{\partial X} &= \frac{\partial L}{\partial y}\mu' W_o^\top = \frac{\partial L}{\partial y_i}\mu' \left[\phi(W_o) + l\right] \\ &\approx \frac{\partial L}{\partial y_i}\mu'(C + A_l B_l^\top)\end{aligned} \tag{5}$$

where C is the target BPN weight, and the initialization residual loss is denoted by $l = W_o^\top - \phi(W_o)$. We compensate the compression loss during initialization by decomposing the l into its low-rank approximation versions, A_l and B_l, with the singular value decomposition (SVD):

$$\begin{aligned} l &\approx \sum \sigma_{t,i} u_{t,i} v_{t,i}^\top \\ A_l &= [\sqrt{\sigma_{t,1}} u_{t,1}, \ldots, \sqrt{\sigma_{t,r}} u_{t,r}] \\ B_l &= [\sqrt{\sigma_{t,1}} v_{t,1}, \ldots, \sqrt{\sigma_{t,r}} v_{t,r}] \end{aligned} \tag{6}$$

where r is the rank of A_l, B_l. With the decoupling approximate initialization, the matrices A_l, B_l serve not only as the bridge between the separate FF and BP pathways but also as the trainable *adapters* for efficient parameter updates during fine-tuning. As shown by Fig. 3(c), along the blue FF path, the output Y is generated through the backbone and adapter. In contrast, following the red BP path, the input gradient G_X is computed through the adapter and BPN, independently of the backbone weights. The backbone weight and BPN weight are frozen during fine-tuning. By adjusting the adapter's rank, we can balance accuracy and computational complexity to achieve different compensation levels during initialization. Based on the FDA, we optimize input gradient computation through the BPN and alleviate the weight gradient computation cost

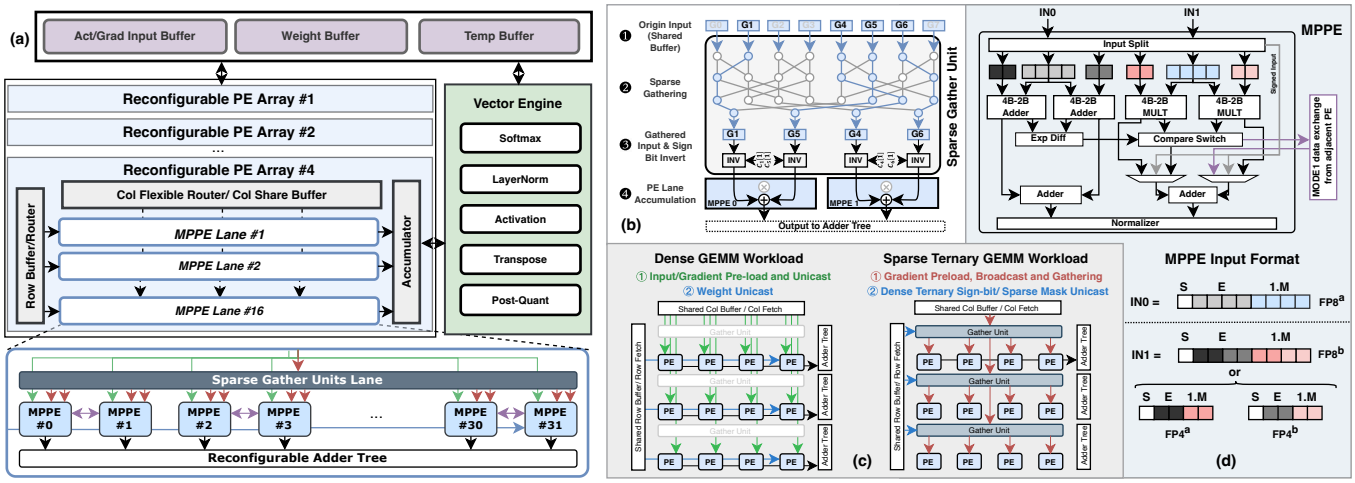

Fig. 4. (a) Overview of DRAFT Accelerator. (b) Illustration of the N:M sparse ternary compute flow in the BPN with an example of $S_t = 0.25$ and $m_t = 8$. (c) Accelerator workflow during the computation of different fine-tuning pathways. *left*: Weight stationary workflow for dense GEMM in backbone and adapter pathway. *right*: Input stationary workflow for sparse ternary GEMM in BPN pathway. (d) The design of MPPE and the input formats.

by the adapter. Collectively, the FDA uniformly reduces the computation overhead of BP, as illustrated in Sec. VI-A.

B. N:M Sparse Ternary Bypass Network

To ensure that the BPN remains lightweight with minimal memory and computational overhead, we employ an N:M sparse ternary quantization method for its initialization [25], [26], as the primary computational burden in BP stems from the floating-point GEMM. As illustrated in Fig. 3(d), this approach eliminates multiplications in the BPN while avoiding substantial routing hardware overheads required by unstructured sparsity, balancing algorithmic and hardware efficiency. We divide the pre-trained weight of each output channel into multiple blocks of size m_t. The BPN ternary weight $c_t^{(i,j,k)}$ for the j-th block in channel i is initialized as follow:

$$c_t^{(i,j,k)} = \begin{cases} 0 & , |w_o^{(i,j,k)}| \le t^{(i,j,:)} \vee k \notin \text{TopK}(|w_o^{(i,j,:)}|) \\ +s^{(i,j,:)} & , w_o^{(i,j,k)} > t^{(i,j,:)} \wedge k \in \text{TopK}(|w_o^{(i,j,:)}|) \\ -s^{(i,j,:)} & , w_o^{(i,j,k)} < -t^{(i,j,:)} \wedge k \in \text{TopK}(|w_o^{(i,j,:)}|) \end{cases} \quad (7)$$

where the $s^{(i,j,:)}$ is the scaling factor of each block. The sparsity ratio of all blocks, denoted as S_t, defines the proportion of non-zero ternary values within the block and is controlled by adjusting the hyper-parameter threshold $t^{(i,j,:)}$ and the TopK selection. The ternary sparsity is determined and encoded at the pre-processing stage.

To further reduce the fine-tuning computation and memory overhead, we employ the widely adopted *Compressing-then-Tuning* scheme [12], [19], and integrate the Microscaling (MX) format [27] to compress each fine-tuning pathway. MX groups m data into a block with a shared exponent (i.e., shared scale 2^s) and normalizes each element in the block with this scale. Each element is either a low bit-width floating point or an integer. We employ mixed-precision MX formats to achieve optimal compression performance, quantizing the pre-trained backbone weights in MXFP4 (E2M1) and the adapter parameters, activations, and gradients in MXFP8 (E4M3). Additionally, the MX block size m is aligned with the N:M ternary quantization block size m_t, both set to 32, ensuring compatibility in hardware implementation.

V. DRAFT Hardware Architecture

A. Overview

Fig. 4(a) illustrates the DRAFT accelerator design, whose primary component is the parallel Reconfigurable Processing Element Arrays (RPEA). Each RPEA contains 16 Mixed-Precision Processing Element (MPPE) lanes, each equipped with 32 MPPEs, 8 Sparse Gather units, and a configurable adder tree. These PEs are interconnected via a dynamic data fetch network. Each RPEA includes row and column ports with local shared buffers and input routers. The pre-compiled configuration enables the scheduling of various workflows to optimize on-chip bandwidth and computational resource utilization. Vector Engines support vector-wise operations such as Softmax, Activation, and Layer Normalization. The total on-chip buffer size is 1 MB. Data is stored across the memory hierarchy using block mapping [28]. The temporary buffer accumulates outputs and supports the Vector Engine. At the start of each batch, weight and input data are pre-loaded into the respective on-chip buffers. DRAFT employs a ping-pong strategy to divide the batch into multiple processing phases if the data exceeds the on-chip buffer capacity.

B. Sparse Gather Unit

To fully leverage the N:M ternary sparsity introduced by the BFN, we design and integrate Sparse Gather units within each PE lane. This unit utilizes an inverse butterfly network [25] to compress gradient data G_Y corresponding to non-zero BFN weights, achieving this in $log(m_t)$ steps. Fig. 4(b) shows the computation process of a 3-stage inverse butterfly network with an input block size of 8 and a BFN sparsity ratio S_t of 0.5. In this setup, gradients with a block size of m_t are broadcast from the column-shared buffer to each gather unit. The gathered gradients are then fetched to the MPPE lane for accumulation directly without multiplication, while their sign bits are adjusted based on the sign bit of dense ternary weight. In the physical design, each lane includes 8 gather units configured with a 32-to-8 structure based on the algorithm sensitivity study in Sec. VI-A.

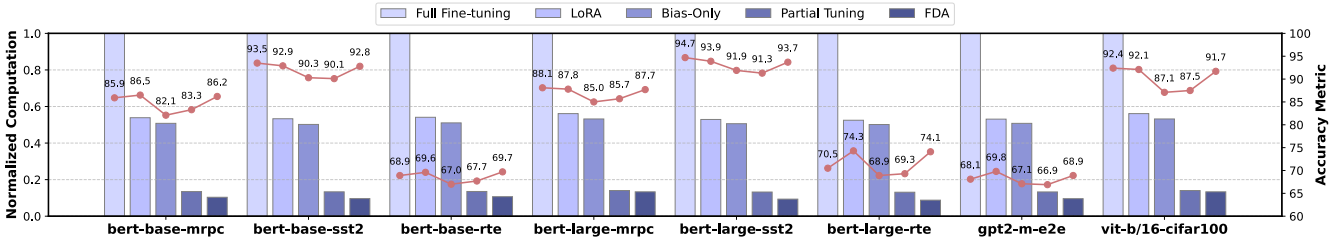

Fig. 5. Fine-tuning accuracy (red dots) and normalized BP computation reduction of different fine-tuning methods on various models and tasks.

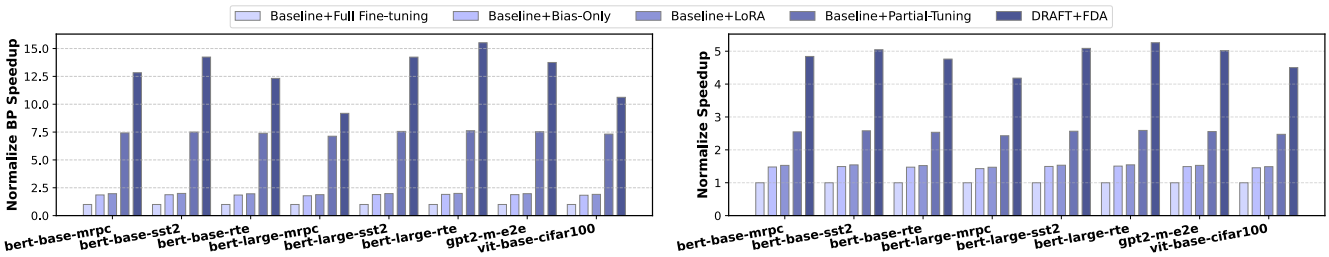

Fig. 6. (a) Normalized BP speedup and (b) normalized end-to-end fine-tuning speedup of DRAFT with FDA, compared to baseline hardware with Full Fine-tuning, LoRA, BitFit, and Partial Tuning (all normalized to Full Fine-tuning running on baseline hardware).

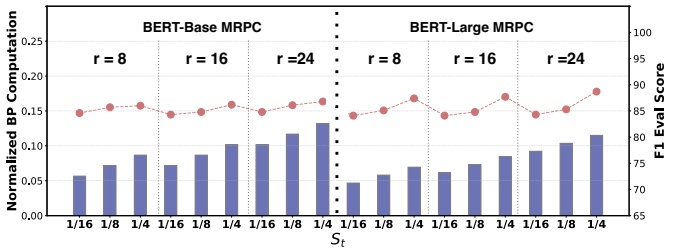

Fig. 7. Fine-tuning accuracy (F1 score metric, red dots) and computation reduction versus various (r, S_t) configurations in FDA.

C. Reconfigurable PE Array

Dual Workflow for Dynamic Fine-tuning Pathway. As illustrated in Fig. 4(c), to fully exploit data parallelism across different fine-tuning pathways, the RPEA can be reconfigured to operate in two distinct workflows: dense GEMM mode and sparse ternary GEMM mode. In the backbone and adapter pathways, all matrix computations are performed in dense mode, utilizing a weight-stationary workflow (shown in Fig. 4(c) left). Weights are pre-loaded into the row buffer and then unicast to each PE, while input data is unicast through a column router to each PE. In contrast, the BPN pathway requires sparse-to-dense gather operations, with each gather unit assigned unique sparsity masks. Sparse ternary matrix multiplication is performed here using an input-stationary workflow (shown in Fig. 4(c) right). Sparse input data is pre-loaded into a column buffer and broadcast to each gather unit, while pre-determined dense ternary sign bits and butterfly control signals are distributed to each gather unit horizontally.

Mixed-Precision Floating-Point PE. The architecture and input format of MPPE is illustrated in Fig. 4(d). The DRAFT fine-tuning process involves three types of matrix operations corresponding to the three modes of the MPPE. *Mode* 0: FP8×FP8, used in the trainable adapter pathway and attention mechanism of the Transformer block; *Mode* 1: FP8×FP4, applied in the pre-trained backbone pathway; *Mode* 2: FP8×Ternary, utilized in the BPN pathway. In Mode 0 and 2, both inputs, IN0 and IN1, are FP8. In Mode 1, IN1 is split into two FP4s to increase the throughput. Each PE receives two FP4 weights corresponding to two output channels, and the adjacent PEs compute the same pair of output channels, exchanging partial results through an inter-PE connection, as illustrated by the purple datapath. The MPPE shares mantissa multipliers and exponent adders across these modes by splitting the MSB and LSB computations.

VI. Evaluation

A. Algorithm Evaluation

Settings: To evaluate the fine-tuning accuracy of FDA, we fine-tuned BERT-Base and BERT-Large [2] models on the MRPC, SST-2, and RTE datasets from GLUE Benchmark [29]; ViT-B/16 model [4] on the CIFAR-100 dataset [30]; and GPT-2 Medium model [31] on the E2E task [32]. Regarding the end-to-end metric, we report the F1 for MRPC, the BLEU for E2E, and the accuracy for the others. We used Full-Finetuning (FT), Bias-Only [8], LoRA, and Partial-Tuning [23] under unified MXFP8 precision as baseline fine-tuning methods. FDA adopts a mixed-precision quantization as introduced in Sec. IV-B. For a fair comparison, the adapter ranks in LoRA, Partial-Tuning, and FDA are set to 16, and all fine-tuning methods have the same hyperparameter setting.

Performance Evaluation: Fig. 5 shows the overall comparison of accuracy metric and normalized BP operations across different fine-tuning methods with S_t in FDA set to 1/4. FT achieves the best accuracy across most tasks but incurs the heaviest BP computational load. LoRA outperformed FT on certain tasks and reduced BP computation to nearly half of FT. The Bias-Only achieved a lighter BP overhead by eliminating weight gradient computation, and Partial-Tuning approached FDA-level reductions by tuning the last few layers. However, both methods exhibited noticeable accuracy losses. In contrast, the FDA maintained an accuracy loss within 1% compared to FT, demonstrating robust performance while reducing BP computation to below 15%. This highlights the effectiveness of the FDA in improving the efficiency of edge fine-tuning.

TABLE I
AREA AND POWER BREAKDOWN OF DRAFT ACCELERATOR (500MHZ)

Module	Area (mm^2)	Power (mW)
RPEA - MPPE & Adder Tree	6.51	729.45
RPRA - Sparse Gather Unit	1.65	142.91
RPEA - Others	1.25	100.28
Vecer Engine	2.01	142.87
SRAM Buffer	2.86	146.06
Others	0.47	26.15
Total	**14.74**	**1287.72**

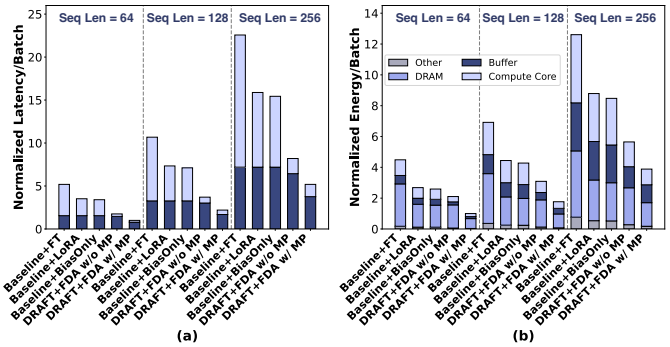

Fig. 8. (a) Latency and (b) Energy breakdown over different hardware with various fine-tuning algorithms for BERT-Base with different batch sequence lengths.

Fig. 7 shows the sensitivity analysis of S_t and r, conducted on BERT-Base and BERT-Large (MRPC). The result indicates that the FDA is more sensitive to BPN sparsity. Notably, when S_t reaches 1/16, a significant accuracy loss is observed, especially in the BERT-Large model. Moreover, a lower S_t incurs additional Sparse Gather unit overhead in hardware implementation. To balance hardware and algorithm efficiency, we keep S_t=1/4 for the following hardware evaluations.

B. Hardware Evaluation

Baseline Setting: We design the baseline hardware based on the SoTA systolic accelerator [18] dedicated to Transformer inference and fine-tuning, which can naturally run all the baseline fine-tuning methods. The baseline hardware is configured to match DRAFT in FP8 compute throughput and memory capacity under the same technology and frequency.

Hardware Implementation: We perform RTL design for the DRAFT and baseline accelerator and synthesized them using Synopsys Design Compiler with a 28nm CMOS technology. The entire DRAFT system operates at 500MHz, with on-chip buffers utilizing foundry-provided SRAM. Power consumption and area results were estimated from the synthesis reports. Additionally, we developed a cycle-accurate simulator and verified it against the RTL implementation. The simulator generates DRAM access traces, used with Ramulator [33] to evaluate DRAM access latency, and DRAMPower [34] to capture the corresponding DRAM power consumption.

Speedup: Fig. 6 illustrates the speedup of DRAFT compared with the baselines. We report both the speedup in the BP phase and the overall fine-tuning speedup. Full-Finetuning, LoRA, Bias-Only, and Partial-Tuning run on the baseline hardware, while FDA runs on DRAFT. During the BP phase, DRAFT achieved an average speedup of 14.1× and 8.3× compared to FT and LoRA running on baseline hardware, respectively. This improvement stems from the N:M sparse ternary BPN, where computational reductions at the algorithmic level align effectively with hardware efficiency. The fine-tuning end-to-end speedup is lower than that of the BP as the DRAFT is tailored to reduce the cost of the BP. DRAFT achieves an average overall speedup of 4.9× by leveraging mixed-precision quantization and the reconfigurable MPPE to improve the throughput of the FF phase.

Hardware Overhead: The hardware area and power consumption of DRAFT after synthesis are shown in Table I. The total power and area are $1.29W$ and $14.74mm^2$, respectively. The MPPE in the RPEA and the buffer significantly contribute to both area and power.

Latency and energy breakdown: Fig. 8 presents the fine-tuning latency and energy breakdown of each batch for BERT-Base with three typical sequence lengths across different fine-tuning algorithms and accelerator hardware. As shown in Fig. 8(a), DRAFT significantly lowers the computational latency of the BP phase, reducing batch latency close to the level of executing only the FF phase. Combined with Mixed-Precision (MP) optimization, DRAFT further reduces latency across all fine-tuning phases.

Fig. 8(b) illustrates the energy breakdown across the compute core, buffer, off-chip memory access, and other modules. By leveraging the FDA, DRAFT achieves 4.2× and 2.3× improvements in energy efficiency compared to FT and LoRA on the baseline hardware, respectively. Based on the algorithm-hardware co-optimization, DRAFT takes full advantage of the parallelism of each fine-tuning pathway, significantly compressing fine-tuning latency and reducing power consumption. Simultaneously, enhanced throughput and improved data reuse alleviate the repeated loading of the substantial weights and batch inputs, thereby lowering the power consumption associated with the expensive on-chip and off-chip data access. In summary, DRAFT significantly improves the throughput and power efficiency of Transformer fine-tuning by fully reducing the BP computational overhead.

VII. CONCLUSION

We propose DRAFT, an algorithm-hardware co-design framework for efficient transformer fine-tuning on edge. The proposed Feedback Decoupling Approximation (*FDA*) algorithm decouples the BP through the backbone weight into the adapter pathway and sparse ternary Bypass Network (*BPN*) pathway. Two lightweight pathways jointly approximate the information of pre-trained weight, uniformly lightening the BP computation overhead for input gradient and weight gradient. An efficient DRAFT accelerator with reconfigurable workflows and sparse gather networks is designed to support fine-tuning with the FDA. Experimental results demonstrate that DRAFT achieves an average speedup of 4.9× and an energy efficiency improvement of 4.2× across multiple fine-tuning tasks with negligible accuracy loss.

REFERENCES

[1] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," May 2019.

[3] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," Jul. 2019.

[4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.

[5] V. Janfaza, S. Mandal, F. Mahmud, and A. Muzahid, "Ada-gp: Accelerating dnn training by adaptive gradient prediction," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1092–1105.

[6] H. Shao, J. Lu, M. Wang, and Z. Wang, "An Efficient Training Accelerator for Transformers With Hardware-Algorithm Co-Optimization," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 11, pp. 1788–1801, Nov. 2023.

[7] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. D. Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-Efficient Transfer Learning for NLP," in *Proceedings of the 36th International Conference on Machine Learning*. PMLR, May 2019, pp. 2790–2799.

[8] H. Cai, C. Gan, L. Zhu, and S. Han, "TinyTL: Reduce Memory, Not Parameters for Efficient On-Device Learning," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 11 285–11 297.

[9] J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar, and Y. Tian, "GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection," Mar. 2024.

[10] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," Oct. 2021.

[11] Y. Li, Y. Yu, C. Liang, P. He, N. Karampatziakis, W. Chen, and T. Zhao, "LoftQ: LoRA-Fine-Tuning-Aware Quantization for Large Language Models," Nov. 2023.

[12] Y. L. H. Y. R. G. X. Z. S. R. B. Y. K. Z. Y. C. L. Zhongzhi Yu, Zheng Wang, "Edge-llm: Enabling efficient large language model adaptation on edge devices via layerwise unified compression and adaptive layer tuning voting," 2024.

[13] Z. Hong and C. P. Yue, "Efficient-grad: Efficient training deep convolutional neural networks on edge devices with grad ient optimizations," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 21, no. 2, pp. 1–24, 2022.

[14] D. Han, J. Lee, and H.-J. Yoo, "Df-lnpu: A pipelined direct feedback alignment-based deep neural network learning processor for fast online learning," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 5, pp. 1630–1640, 2021.

[15] M. Giordano, K. Prabhu, K. Koul, R. M. Radway, A. Gural, R. Doshi, Z. F. Khan, J. W. Kustin, T. Liu, G. B. Lopes, V. Turbiner, W.-S. Khwa, Y.-D. Chih, M.-F. Chang, G. Lallement, B. Murmann, S. Mitra, and P. Raina, "Chimera: A 0.92 tops, 2.2 tops/w edge ai accelerator with 2 mbyte on-chip foundry resistive ram for efficient training and inference," in *2021 Symposium on VLSI Circuits*, 2021, pp. 1–2.

[16] L. Zhu, L. Hu, J. Lin, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "Pockengine: Sparse and efficient fine-tuning in a pocket," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1381–1394.

[17] K. Prabhu, R. M. Radway, Y. Jeffrey, K. Bartolone, M. Giordano, F. Peddinghaus, Y. Urman, W.-S. Khwa, Y.-D. Chih, M.-F. Chang, S. Mitra, and P. Raina, "Minotaur: An edge transformer inference and training accelerator with 12 mbytes on-chip resistive ram and fine-grained spatiotemporal power gating," in *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, 2024, pp. 1–2.

[18] J. Yu, K. Prabhu, Y. Urman, R. M. Radway, E. Han, and P. Raina, "8-bit Transformer Inference and Fine-tuning for Edge Accelerators," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. La Jolla CA USA: ACM, Apr. 2024, pp. 5–21.

[19] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[20] T. H. Moskovitz, A. Litwin-Kumar, and L. Abbott, "Feedback alignment in deep convolutional networks," *arXiv preprint arXiv:1812.06488*, 2018.

[21] Q. Liao, J. Leibo, and T. Poggio, "How important is weight symmetry in backpropagation?" in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[22] J. Launay, I. Poli, F. Boniface, and F. Krzakala, "Direct feedback alignment scales to modern deep learning tasks and architectures," *Advances in neural information processing systems*, vol. 33, pp. 9346–9360, 2020.

[23] R. Zhang, J. Han, A. Zhou, X. Hu, S. Yan, P. Lu, H. Li, P. Gao, and Y. J. Qiao, "Llama-adapter: Efficient fine-tuning of language models with zero-init attention," *ArXiv*, vol. abs/2303.16199, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:257771811

[24] Y.-L. Sung, J. Cho, and M. Bansal, "Lst: Ladder side-tuning for parameter and memory efficient transfer learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 12 991–13 005, 2022.

[25] S. Liu, P. Li, J. Zhang, Y. Wang, H. Zhu, W. Jiang, S. Tang, C. Chen, Q. Liu, and M. Liu, "16.2 a 28nm 53.8tops/w 8b sparse transformer accelerator with in-memory butterfly zero skipper for unstructured-pruned nn and cim-based local-attention-reusable engine," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, pp. 250–252.

[26] C. Fang, A. Zhou, and Z. Wang, "An algorithm–hardware co-optimized framework for accelerating n: M sparse transformers," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 11, pp. 1573–1586, 2022.

[27] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf *et al.*, "Microscaling data formats for deep learning," *arXiv preprint arXiv:2310.10537*, 2023.

[28] S. Q. Zhang, T. Tambe, N. Cuevas, G.-Y. Wei, and D. Brooks, "CAMEL: Co-Designing AI Models and Embedded DRAMs for Efficient On-Device Learning," Dec. 2023.

[29] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "Glue: A multi-task benchmark and analysis platform for natural language understanding," *arXiv preprint arXiv:1804.07461*, 2018.

[30] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[31] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[32] J. Novikova, O. Dušek, and V. Rieser, "The e2e dataset: New challenges for end-to-end generation," *arXiv preprint arXiv:1706.09254*, 2017.

[33] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.

[34] K. Chandrasekar, C. Weis, Y. Li, B. Akesson, N. Wehn, and K. Goossens, "Drampower: Open-source dram power & energy estimation tool," *URL: http://www. drampower. info*, vol. 22, 2012.